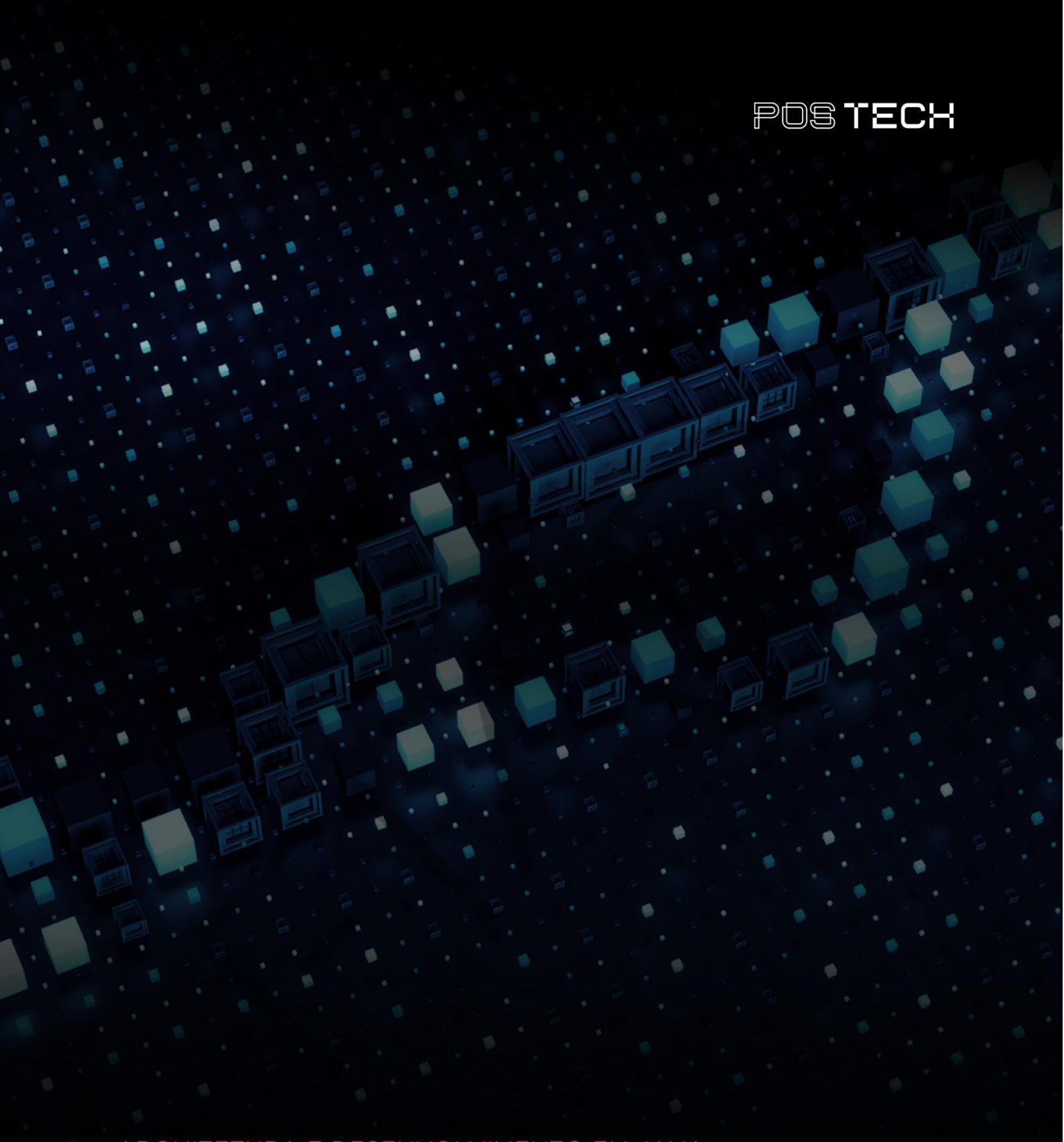


POS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 4 | AULA 05 -

GERENCIAMENTO DE CUSTOS E PERFORMANCE

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ?.....3
HANDS ON 4
SAIBA MAIS.....5
O QUE VOCÊ VIU NESTA AULA? 13
REFERÊNCIAS.....14

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Citada como uma das grandes vantagens do uso de serverless, a redução de custos nem sempre acontece como esperado. Por isso, vamos entender o custo financeiro de se aplicar a arquitetura serverless para implementarmos uma solução com custo adequado, que funcione sem grandes sustos na fatura mensal. Vamos também abordar a importância do desempenho da solução e qual seu impacto na operação.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

O serverless tem sido uma força motriz no surgimento de startups de tecnologia, dado que seu custo inicial é muito pequeno, cobrado por uso. Essa mesma característica se torna problemática em projetos de tecnologia que precisam de previsibilidade orçamentária ou têm orçamento restrito.

Existem modos adequados de entender e lidar com essas questões financeiras, coordenando conhecimento técnico, uso de ferramentas disponíveis e alinhamento com as áreas de negócio para distribuir o custo de usar esta abordagem.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

Quanto custa ser serverless?

Ao adotar arquiteturas serverless, uma das primeiras promessas é a eliminação completa do custo associado à infraestrutura ociosa (CAPEX). Diferentemente dos modelos tradicionais baseados em servidores (sejam físicos, máquinas virtuais ou containers sempre ativos), no serverless você pagará, essencialmente, **por uso real**, que pode ser medido em requisições, tempo de execução, volume de dados processados ou eventos disparados.

Se pensarmos no curto prazo ou em uma operação em início de vida, ainda tentando se provar como viável, essa mudança de modelo traz uma enorme atratividade. Ela é percebida como vantajosa especialmente para projetos de pequeno porte e startups. No entanto, a mesma mudança que oferece escalabilidade automática e eficiência operacional também introduz uma nova camada de complexidade: **a imprevisibilidade dos custos**.

Vamos pensar em alguns fatores que impactam diretamente a previsão dos custos:

• **Múltiplas métricas de faturamento:** o custo não se baseia apenas no tempo de execução. Fatores como quantidade de memória alocada para uma função, número de solicitações, transferência de dados e uso de serviços auxiliares (como APIs de gateway, filas de mensagens e bancos de dados serverless) impactam diretamente na complexidade de prever o custo de operação de um dado componente.

• **Tráfego imprevisível:** se aplicações orientadas a eventos e/ou com picos de demanda imprevisíveis são candidatas ideais para serverless, elas também são justamente as mais difíceis de prever em termos de custo. Um evento viral nas redes sociais ou uma campanha de marketing bem-sucedida podem levar a um aumento exponencial e inesperado nos custos, muitas vezes até reduzindo o ROI que esses eventos podem proporcionar.

• **Complexidade da aplicação:** em uma arquitetura de microsserviços baseada em serverless, uma única transação de usuário pode desencadear uma cascata de invocações de funções, tornando o rastreamento do custo por transação uma tarefa árdua.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Se consideramos os modelos tradicionais on premises, com data center local ou delegado, ou cloud com instâncias fixas, a previsão de custos é praticamente direta. O custo está associado ao investimento de aquisição e ao custo de operação dos recursos provisionados: CPU, memória, armazenamento e rede, independentemente do uso.

Agora, vamos pensar tecnicamente nos pontos citados. No modelo serverless, este custo é **granular** e está diretamente associado a variáveis como:

• Quantidade de execuções de funções ou serviços.

• Duração das execuções (em milissegundos ou segundos).

• Volume de dados transferidos ou processados.

• Número de eventos gerados (em pub/sub, filas, gatilhos de storage etc.).

• Chamadas a APIs, tanto internas quanto externas.

Em cloud, cada pequena ação implica em um custo, que na maioria das vezes é tido como “ínfimo”, “marginal” ou “desprezível”. Vejamos um exemplo um pouco ingênuo, mas que pode nos ajudar na análise, baseado na calculadora de custos da AWS:

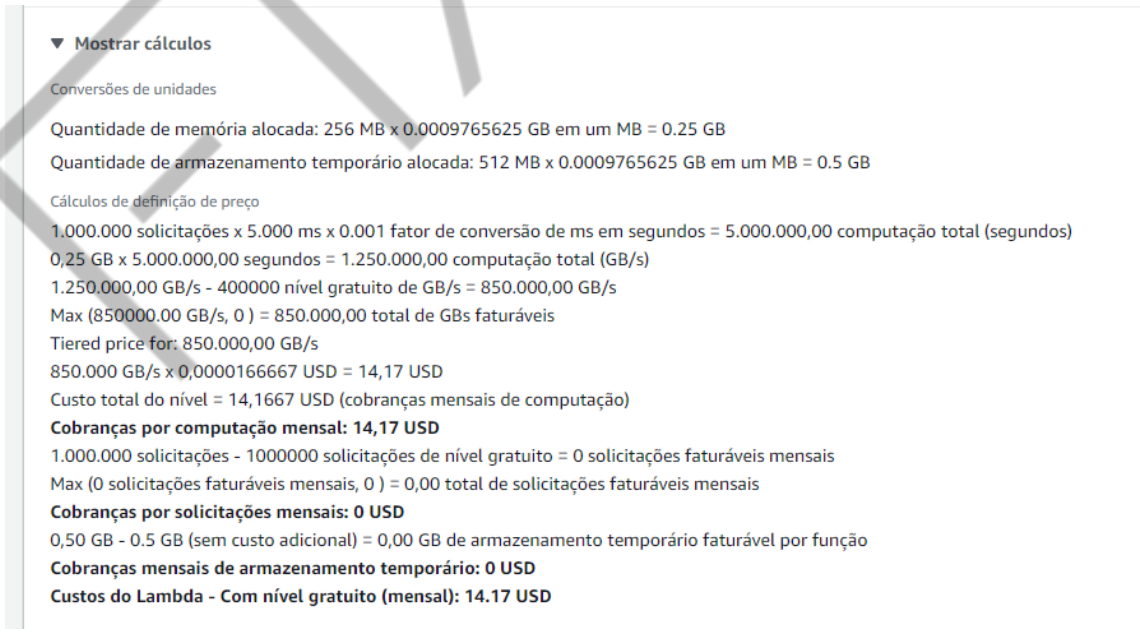


Figura 1 - Custo de operação de serverless, verificado em junho de 2025
Fonte: elaborado pelo autor (2025)

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Temos que cada chamada à função, dado a configuração informada e detalhada na figura, custa USD 0,0014147. Como estamos prevendo 1 milhão de chamadas, temos o custo de USD 14,17.

Isso significa que o custo se comporta em função do **tráfego**, **uso** e **complexidade operacional da aplicação**, o que pode gerar surpresas tanto em subutilização (custo extremamente baixo) quanto em explosões inesperadas de uso e, portanto, de custo.

Voltando ao exemplo acima, se mantivermos a ideia desse cálculo, um aumento de dez vezes em chamadas aumenta em dez vezes o preço a ser pago. Muitas vezes, esse tipo de aumento é imprevisível. Se o custo de cada uma dessas chamadas está embutido no custo da operação e cada chamada de função se refere a um processo de negócio que retorna um valor direto associado a esta execução (uma confirmação de venda, por exemplo), o aumento do custo é, de certa forma, absorvido.

Até mesmo pensando em uma operação que cresce de forma orgânica, com aumento de usuários, eventos e interações, temos que analisar se o uso de serverless é realmente vantajoso, pois pode gerar uma curva de custo muito mais inclinada do que em modelos tradicionais, especialmente quando existem processos assíncronos que se multiplicam na retaguarda (ex.: filas, triggers, pipelines).

Decisões arquiteturais e custo

Se serverless muda o paradigma do controle de custos, mostrando que agora o preço a pagar é calculado **por operação**, o foco da arquitetura muda. Não queremos mais saber “Quantas máquinas eu preciso?”, e sim *“*Quantos eventos meu sistema gera?”, “Quanto tempo minhas funções levam para processar cada evento?” e “Qual é a eficiência do meu código?”

O controle de custo, portanto, está intrinsecamente ligado:

• **Design da arquitetura:** excesso de decomposição pode gerar custos desnecessários (ex.: muitas funções executando pequenas partes, com overhead elevado em chamadas entre si).

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

• **Qualidade do código:** funções ineficientes, com má gestão de memória ou loops desnecessários, podem custar significativamente mais, pois o tempo de execução impacta diretamente o faturamento.

• **Gestão de eventos:** pipelines excessivamente fragmentados, filas que disparam múltiplas execuções ou padrões mal desenhados (como polling desnecessário) podem multiplicar os custos.

Se o projeto da arquitetura não levar esses pontos em consideração, há o risco real de que a aplicação se torne, com o tempo, um ponto de drenagem financeira, quando considerarmos o custo de operação (OPEX).

FinOps

Vimos que, à medida que a aplicação cresce em quantidade de operação (seja em adoção e/ou volume), o custo também cresce de forma proporcional ou, muitas vezes, desproporcional, dependendo da arquitetura projetada.

Os principais desafios, então, consistem em identificar quais partes do sistema estão consumindo mais recursos, separar custos por serviço, time ou funcionalidade e detectar padrões de crescimento de custo que não estão acompanhando o crescimento do negócio.

Muitos provedores oferecem ferramentas básicas de análise de custo (ex.: AWS Cost Explorer, Google Cloud Billing, Azure Cost Management), mas, no serverless, essas ferramentas nem sempre oferecem granularidade suficiente para entender, por exemplo, qual função lambda consome mais em termos de I/O vs. CPU ou, então, se uma explosão de custos veio de um aumento real de tráfego ou de um bug que disparou loops infinitos ou retries automáticos.

Esse nível de controle, cada vez mais necessário, demanda **instrumentação adicional**, com ferramentas específicas de observabilidade, rastreamento distribuído e monitoramento financeiro detalhado. Diante desse cenário, entra o conceito de **FinOps**, uma prática que une **finanças, operações e engenharia/arquitetura**, permitindo que as organizações tomem decisões informadas sobre o custo na nuvem.

No contexto serverless, FinOps se traduz em práticas como:

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

• **Identificação e compartimentação de recursos:** todas as funções, tópicos, filas e buckets devem ser identificadas (tagging) por contextos como produto, time ou ambiente.

• **Métricas de custo por unidade de negócio:** transformar o custo técnico (em gigabytes, execuções ou milissegundos) em custo por usuário, por transação ou por cliente.

• **Alertas proativos de custo:** monitorar o custo como se monitora o desempenho de uma aplicação. Por exemplo, se uma função começa a gerar um custo 20% acima da média, um alerta dispara antes que a fatura estoure.

• **Acompanhamento em tempo real do custo variável:** dashboards que mostram como os custos estão evoluindo no dia, na semana ou no mês.

• **Otimização contínua:** refinar código, repensar arquitetura (ex.: mover partes altamente intensivas para containers serverless, como Cloud Run ou Fargate, se for mais barato), e evitar antipadrões de uso.

• **Colaboração interdepartamental:** integrar as equipes de tecnologia, finanças e negócios para alinhar os investimentos em nuvem com os objetivos da empresa.

• **Capacitar os times para tomada de decisões arquiteturais baseadas também em custo de operação:** capacitar as equipes de desenvolvimento a tomar decisões de arquitetura e otimização com base no impacto financeiro. O time de desenvolvimento não precisa apenas saber como o código funciona, mas também quanto ele custa para ser executado.

Performance importa?

Em modelos serverless, o custo de execução está diretamente vinculado ao tempo de processamento e ao consumo de recursos. Portanto, **pagamos por cada execução.** Funções mais lentas não afetam só a experiência do usuário, mas também impactam os custos operacionais. Há, ainda, o modelo de escalabilidade automática, que faz com que problemas de desempenho se multipliquem proporcionalmente ao aumento da demanda.

Existem também alguns desafios inerentes ao construir software para que seja executado como serviço. Por exemplo, sua **natureza distribuída e efêmera**.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Aplicações são compostas por um conjunto de funções pequenas, independentes e que executam por um curto período. O comportamento do ambiente de execução (runtime) não pode ser ajustado, e rastrear o desempenho de uma única transação que “atravessa” múltiplas funções, filas e fontes de dados exige ferramentas de observabilidade e tracing distribuído robustas.

Além disso, parte da infraestrutura de execução é uma **caixa-preta gerenciada pelo provedor de nuvem**. Compreender gargalos que ocorrem nesse nível, como a latência de rede interna ou a contenção de recursos na infraestrutura compartilhada, é complexo e, na maioria das vezes, impossível. Um exemplo já citado é o **cold start**, que não é linear e não existe um modelo que defina exatamente quando uma execução vai precisar desse “tempo extra” de alocação e inicialização.

Ao mesmo tempo que não conseguimos monitorar diretamente cada execução de forma antecipada, temos uma **quantidade enorme de métricas** que podem ser usadas para analisar a execução a posteriori, muitas delas extremamente especializadas em um único indicador (como quantidade de processamento realmente usado, tempo de execução de código ou tempo de ociosidade). Pensando em uma solução com uma quantidade grande de funções e cada função gerando um conjunto próprio de métricas (invocações, duração, erros, uso de memória), a quantidade de dados a serem analisados pode ser esmagadora, tornando difícil a identificação de sinais em meio ao ruído. Até mesmo o entendimento de **quais dados são realmente úteis em cada contexto de análise** pode ser um trabalho não apenas desafiador, mas também baseado em tentativa e erro, buscando extrair de dados diferentes algum resultado que faça sentido no contexto necessário.

Monitorar desempenho em serverless é, por si só, um desafio. Não existem instâncias ou servidores dedicados para observar, porque cada execução é efêmera (por definição), distribuída (com execução simultânea e, às vezes, concorrendo por recursos externos com outras execuções) e isolada. As ferramentas tradicionais de APM (Application Performance Monitoring) precisam ser adaptadas para capturar métricas como:

- **Cold Start:** tempo necessário para inicializar uma função inativa.
- **Throughput de concorrência:** concorrência em funções espalhadas por múltiplos nós.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

• **Latência das dependências:** impacto temporal da espera pela resposta a chamadas a serviços externos, como bancos de dados, filas e APIs.

• **Erros intermitentes e/ou degradação sob carga:** muitas vezes mascarados em execuções isoladas.

Tem saída?

Sim, fazendo o possível com as ferramentas que temos e, principalmente, com acesso aos principais insumos de análise, que são o código-fonte e a configuração desejada para a execução desse código com função. Usando esses insumos, combinados aos dados de monitoramento, conseguimos atuar em um fine-tuning dos componentes que envolvem código-fonte. Usualmente, os principais pontos de melhoria são:

1. Alocação de recursos: em serviços de execução de funções, **a alocação de memória está diretamente atrelada ao poder de CPU**. Alocar mais memória pode reduzir drasticamente o tempo de execução de uma função computacionalmente intensiva, o que, por sua vez, pode diminuir o custo total da invocação. Ferramentas de power tuning de cada linguagem de programação automatizam o processo de encontrar o ponto ótimo nessa curva de “custo vs. performance”. Por exemplo, usando containers lxc com recursos limitados é possível tentar reproduzir o comportamento do código em um ambiente restrito em relação a processador e memória.

2. Gerenciamento de concorrência: entender como a concorrência impacta o desempenho geral médio da execução de uma função. Uma vez que isso é identificado como um problema, temos dois tipos de solução:

- **Concorrência reservada:** garante que uma função crítica sempre tenha um número mínimo de instâncias disponíveis, evitando que outras funções consumam toda a capacidade da conta e causem throttling. Ao mesmo tempo, esse tipo de configuração pode limitar o número máximo de instâncias concorrentes.
- **Concorrência provisionada**: mantém um número específico de instâncias da função “aquecidas” e prontas para executar, eliminando virtualmente o cold start. Essa é uma ferramenta poderosa para aplicações sensíveis à latência, mas possui um custo fixo, contrariando o modelo “pague pelo uso”, e que pode ser proibitiva. Nesse caso, é

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

importante questionar se essa funcionalidade realmente precisa estar em uma estrutura de função como serviço.

3. Otimização de código e dependências: a performance começa no código. Reduzir o tamanho do pacote de deploy, minimizar dependências, inicializar conexões de banco de dados fora do handler da função, trabalhar com acesso a origens de dados externas otimizadas e escolher runtimes mais eficientes são práticas que diminuem tanto o tempo de execução quanto a latência do cold start.

Então, a performance importa? **Sim, mas seu grau de importância é definido pelo contexto do negócio.**

Para um endpoint de API que serve uma aplicação de venda de produtos, cada átimo de tempo impacta na retenção do cliente, cada milissegundo de latência pode impactar a experiência do usuário e as taxas de conversão. Nesses casos, investir em concorrência provisionada e fine-tuning agressivo é justificável.

Mas, se considerarmos um processo de backend assíncrono que processa dados em lote durante a madrugada, uma latência de alguns segundos em um cold start pode ser perfeitamente aceitável e a prioridade se desloca para o menor custo de execução possível.

O papel da pessoa que pensa a arquitetura de software usando serverless se mostra, então, menos sobre alcançar a performance máxima a todo custo e mais sobre entender e calibrar os trade-offs. O melhor desempenho é o adequado e o que significa “adequado” muda de acordo com a realidade. Isso passa por conhecer as necessidades de negócio, os custos que cada decisão de arquitetura implica e qual decisão tomar para atender ao processo de negócio. Em serverless, a performance não é apenas uma característica técnica; é uma dimensão fundamental da estratégia de produto e financeira.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, entendemos os custos associados à adoção de uma arquitetura serverless, bem como a importância da performance e seus custos.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

AWS. **AWS Well-Architected**. Disponível em: https://aws.amazon.com/pt/architecture/well-architected/. Acesso em: 07 jul. 2025.

GOOGLE Cloud. **Modelo de arquitetura do Google Cloud**. Disponível em https://cloud.google.com/architecture/framework?hl=pt-br. Acesso em: 07 jul. 2025. 06/2025

INDRASIRI, K; SUHOTHAYAN, S. **Design Patterns for Cloud Native Applications**. [S.l.]: O'Reilly, 2021.

MICROSOFT. **Azure Architecture Center**. Disponível em https://learn.microsoft.com/en-us/azure/architecture. Acesso em: 07 jul. 2025.

NEWMAN, S. **Building Microservices**: Designing Fine-Grained Systems. [S.l.]: O'Reilly, 2021.

FIAP

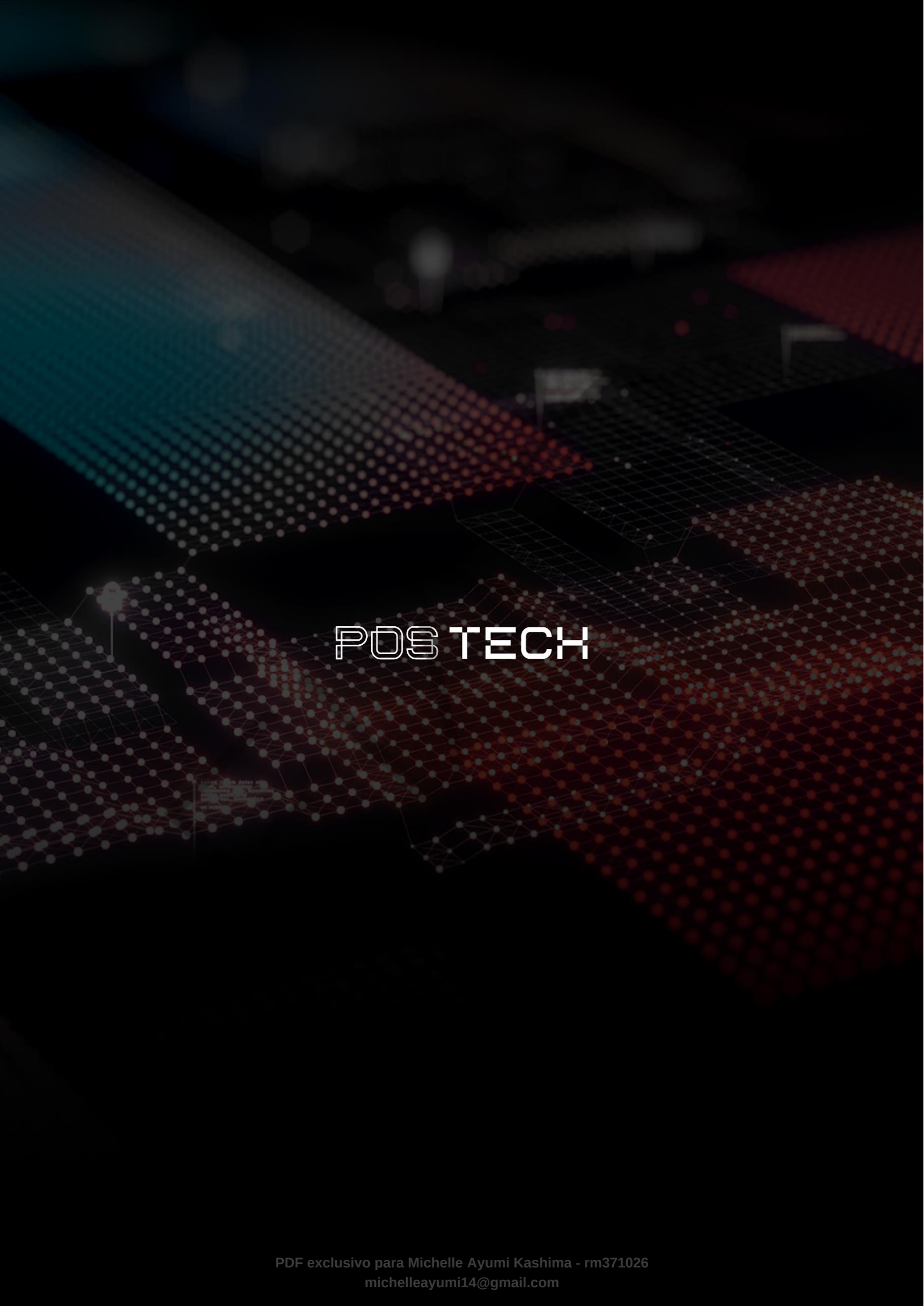
PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Serveless. Custos e performance.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com